m y _ s h i f t r e g _ 1 0 1 0 1 1 1 _ d e t e c t
my_fsm_1010111_detect 구현 하기

source code

my_shiftreg_1010111_detect.v

```
`timescale 1ns / 1ps

module my_shiftreg_1010111_detect(
    input clk,
    input reset,
    input btn_press,
    input x,
    output [15:0] led
    );

    // 버튼을 눌렀다 뗀 순간만 shift

    reg [6:0] sr7;

    always @(posedge clk, posedge reset) begin
        if(reset) begin
            sr7 <= 7'b0000000;
        end else if (btn_press)
                    sr7 <= {sr7[5:0], x}; // shift register
    end

    assign led[6:0] = sr7;
    assign led[14:7] = 8'b0;
    assign led[15] = (sr7 == 7'b1010111) ? 1 : 0;
endmodule
```

my_fsm_1010111_detect.v

```
`timescale 1ns / 1ps

module my_fsm_1010111_detect(
    input clk,
    input reset,
```

```
    input btn_press,
    input x,
    output [15:0] led
    );

    // 상태 인코딩 (S0:대기 S1:1검출 S2:10검출 S3:101검출)
    localparam [2:0] S0 = 3'b000,       // IDLE
                     S1 = 3'b001,       // 1검출
                     S2 = 3'b010,       // 10검출
                     S3 = 3'b011,       // 101검출
                     S4 = 3'b100,       // 1010검출
                     S5 = 3'b101,       // 10101검출
                     S6 = 3'b110,       // 101011검출
                     S7 = 3'b111;       // 1010111 검출

    reg [6:0] curr_state, next_state;

    // 1. curr state 상태 register (순차회로)
    always @(posedge clk, posedge reset) begin
        if (reset) curr_state <= S0;
        else curr_state <= next_state;
    end

    // 2. 다음 상태 로직 (조합회로)
    always @(*) begin
        next_state = curr_state;   // !btn_press면 현재 상태 유지
         if(btn_press) begin    // btnU나 btnD 둘중 1개를 누른 시점에만 상태
천이
            case (curr_state)
            S0: next_state = x ? S1 : S0;
            S1: next_state = x ? S1 : S2;
            S2: next_state = x ? S3 : S0;
            S3: next_state = x ? S1 : S4;
            S4: next_state = x ? S5 : S0;
            S5: next_state = x ? S6 : S4;
            S6: next_state = x ? S7 : S2;
            S7: next_state = x ? S1 : S2;    // 중첩 허용
            default:  next_state = S0;   // 래치 방지
            endcase
```

```
        end
    end

    // 3. 출력 로직 : 현재 상태로만 출력 (moore machine)
    assign led[2:0] = curr_state;
    assign led[9:3] = (curr_state == S7) ? 7'b1010111 : 7'b0000000;  // 상
태 확인용
    assign led[14:10] = 5'b0;
    assign led[15] = (curr_state == S7);
endmodule
```

test bench code

```
`timescale 1ns / 1ps

module tb_shift_register();

    reg clk = 0;
    reg reset = 0;
    reg btn_press = 0;
    reg x = 0;

    wire [15:0] led;

    // 인스턴스 생성
    my_shiftreg_1010111_detect  uut(
        .clk(clk),
        .reset(reset),
        .btn_press(btn_press),
        .x(x),
        .led(led)
    );

    // clk 설정
    always #5 clk = ~clk;  // 100MHz

    // 데이터 1비트 입력
    task send_data(input data);
    begin
```

```
        @(negedge clk);
        x = data;
        btn_press = 1;

        @(negedge clk);
        btn_press = 0;
    end
    endtask

    initial begin
        reset = 1;
        #20;
        reset = 0;

        // 1010111 입력
        send_data(1);
        send_data(0);
        send_data(1);
        send_data(0);
        send_data(1);
        send_data(1);
        send_data(1);

        #1;
        // 1010111이 입력되었을 때 led[15] on
        if (led[6:0] == 7'b1010111 && led[15] == 1)
            $display("TEST PASS!");
        else
            $display("TEST FAIL! led=%b", led);

        #20;
        $finish;
    end

endmodule
```

test bench 결과

